

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчёт

Практическая работа

“ДЗ4: Технический дизайн микросервисной архитектуры”

Выполнил:

Шестак Б.Е.

К3340

Проверил:

Добряков Д.И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное бэкенд-приложение «Платформа поиска работы» на микросервисы, придерживаясь концепции database-per-service. Спроектировать микросервисную архитектуру, описать способы взаимодействия сервисов, разделение баз данных, спроектировать эндпоинты для межсервисного обмена в формате OpenAPI. Сформировать план перехода по паттерну Strangler Fig.

1. Анализ текущего монолита

Существующее приложение (результат ЛР1) реализовано как монолит на связке FastAPI + SQLAlchemy + SQLite. Все девять таблиц базы данных находятся в единой схеме, все эндпоинты обрабатываются одним сервером.

Таблицы монолита:

- users — пользователи (соискатели и работодатели)
- skills — справочник навыков
- companies — компании работодателей
- vacancies — вакансии
- vacancy_skills — связь M:N вакансий и навыков
- resumes — резюме соискателей
- work_experience — опыт работы
- education — образование
- applications — отклики соискателей на вакансии

Основные группы эндпоинтов: аутентификация (регистрация, вход, обновление токенов), профиль пользователя, справочник навыков,

управление компаниями и вакансиями, резюме с опытом и образованием, отклики соискателей.

Недостатки монолитного подхода: невозможность независимого масштабирования, единая точка отказа, высокая связность модулей.

2. Декомпозиция на микросервисы

На основе анализа предметной области и структуры базы данных выделены четыре микросервиса, каждый из которых владеет собственным набором таблиц и предоставляет изолированный функционал.

Таблица 1 — Состав микросервисов

Микросервис	Порт	База данных	Таблицы	Функционал
auth-service	8001	auth.db	users	Регистрация, вход, обновление токенов, профиль пользователя
vacancy-service	8002	vacancy.db	skills, companies, vacancies, vacancy_skills	Управление каталогом вакансий, компаниями и справочником навыков
resume-service	8003	resume.db	resumes, work_experience, education, resume_skills	Резюме соискателей, опыт работы, образование, навыки резюме
application-service	8004	application.db	applications	Создание откликов на вакансии, история, смена статуса

Обоснование границ:

auth-service изолирован по соображениям безопасности, содержит критичные данные (пароли, токены), редко изменяется.

vacancy-service объединяет операции с каталогом вакансий, которые преимущественно являются читательскими. Навыки и компании — вспомогательные сущности, неотделимые от вакансий.

resume-service отвечает за личный кабинет соискателя. Резюме, опыт работы и образование тесно связаны и всегда запрашиваются вместе.

application-service изолирует транзакционную логику откликов. Вынесен отдельно для независимой обработки пиковых нагрузок в периоды массовых рекрутинговых кампаний.

3. Атрибуты и связи внутри микросервисов

auth-service (база auth.db)

Содержит единственную таблицу users с полями: user_id (PK), email (уникальный), hashed_password, role (applicant/employer), first_name, last_name, phone, created_at. Связей с другими таблицами внутри сервиса нет.

vacancy-service (база vacancy.db)

Содержит четыре таблицы. Центральная — vacancies, к которой привязана зависимая таблица через внешний ключ company_id.

Справочник skills связан с vacancies через ассоциативную таблицу vacancy_skills (M:N). Таблица companies хранит owner_id — идентификатор пользователя из auth-service без ограничения внешнего ключа.

resume-service (база resume.db)

Содержит четыре таблицы. Центральная — resumes (user_id ссылается на auth-service). К ней привязаны work_experience и education через resume_id (FK внутри сервиса). Таблица resume_skills хранит skill_id из vacancy-service без FK.

application-service (база application.db)

Содержит единственную таблицу applications. Поля vacancy_id, user_id, resume_id — идентификаторы из других сервисов, хранятся как обычные числовые поля без ограничений целостности. Целостность проверяется через HTTP.

4. Межсервисное взаимодействие

Взаимодействие между микросервисами осуществляется синхронно через HTTP REST. Каждый сервис предоставляет внутренние эндпоинты, доступные только внутри сети микросервисов.

Таблица 2 — Межсервисные связи

От (сервис)	К сервису	Эндпоинт	Назначение
vacancy-service	auth-service	GET /internal/validate	Проверка JWT при создании вакансии
resume-service	auth-service	GET /internal/validate	Проверка JWT при работе с резюме
application-service	auth-service	GET /internal/validate	Проверка JWT при создании отклика
application-	vacancy-	GET	Проверка

От (сервис)	К сервису	Эндпоинт	Назначение
service	service	/internal/vacancies/{id}	существования вакансии и её активности
application- service	resume-service	GET /resumes/internal/{id}	Проверка резюме и принадлежности пользователю

5. Спецификация внутренних эндпоинтов (OpenAPI)

auth-service (порт 8001):

GET /internal/validate?token=<jwt> — проверка токена. Возвращает { user_id, role, email } (200) или { detail: "Invalid token" } (401).

vacancy-service (порт 8002):

GET /internal/vacancies/{vacancy_id} — проверка вакансии. Ответ: 200 с полными данными вакансии или 404 с описанием ошибки.

resume-service (порт 8003):

GET /resumes/internal/{resume_id} — проверка резюме. Ответ: 200 с данными резюме (включая user_id) или 404.

Единый формат ошибок:

```
{
  "detail": "описание ошибки"
}
```

Таблица 3 — Коды ответов внутренних эндпоинтов

Код	Когда возникает
200 OK	Ресурс найден, данные возвращены
401 Unauthorized	Токен недействителен или истёк
404 Not Found	Запрошенный ресурс не существует
503 Service Unavailable	Целевой сервис недоступен при HTTP-вызове

6. Разделение баз данных

Каждый микросервис использует собственную базу данных SQLite. Это обеспечивает независимость развёртывания и отсутствие разделяемых таблиц.

- auth-service → база auth.db (таблица users)
- vacancy-service → база vacancy.db (таблицы skills, companies, vacancies, vacancy_skills)
- resume-service → база resume.db (таблицы resumes, work_experience, education, resume_skills)
- application-service → база application.db (таблица applications)

Внешние ключи между базами данных не используются. Целостность данных обеспечивается на уровне приложения через описанные выше HTTP-запросы.

7. Схема микросервисной архитектуры

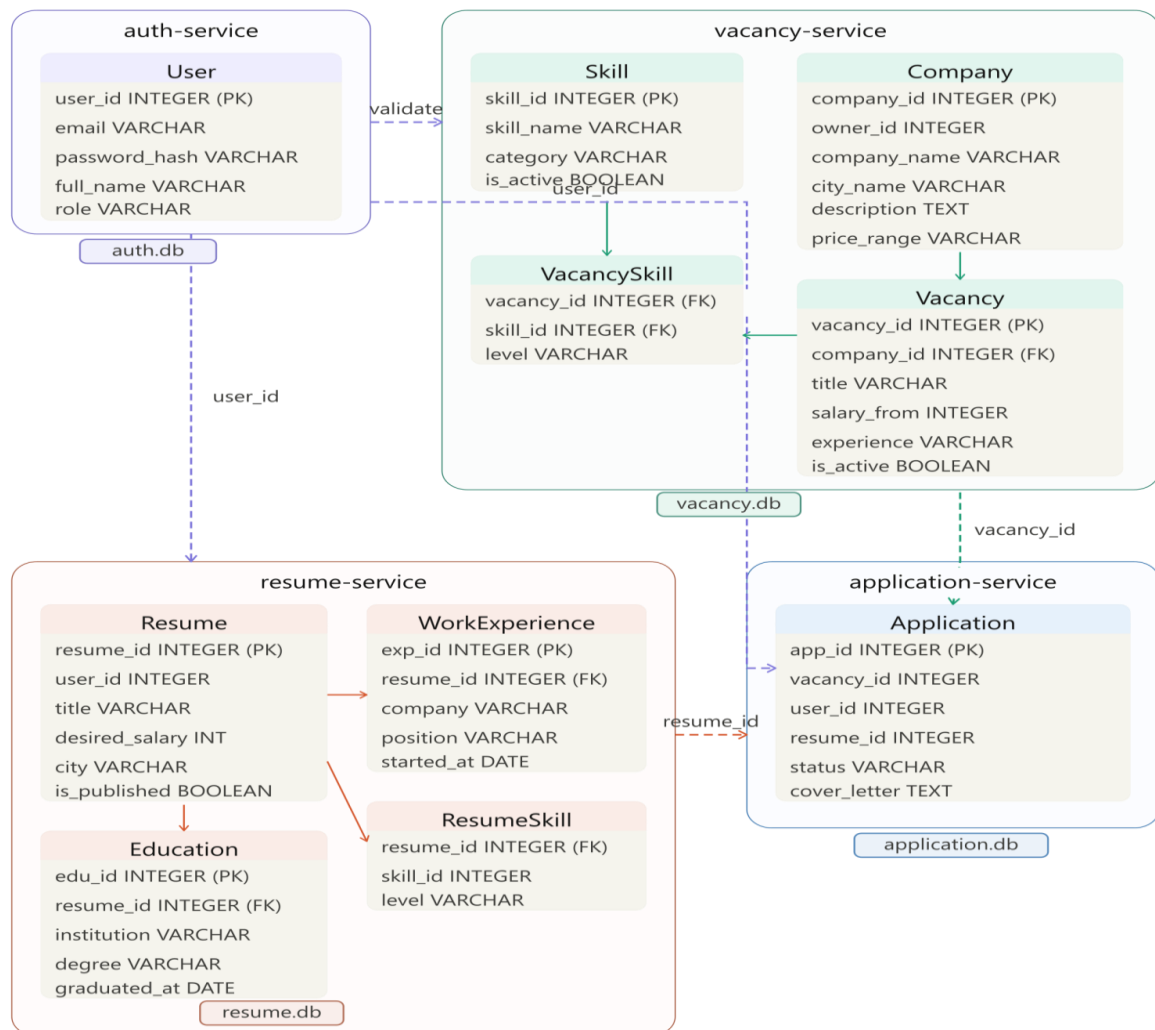


Схема включает четыре микросервиса с их таблицами, внутрисервисными связями и межсервисными стрелками через HTTP. Базы данных изображены отдельно для каждого сервиса.

На схеме отражены:

- границы микросервисов (`auth-service`, `vacancy-service`, `resume-service`, `application-service`)
- таблицы внутри каждого сервиса с атрибутами и первичными ключами

- внутрисервисные связи (vacancies → vacancy_skills, resumes → work_experience, resumes → education)
- межсервисные связи (applications → users, applications → vacancies, applications → resumes)
- отдельные базы данных для каждого сервиса

8. Способы взаимодействия

Все сервисы взаимодействуют синхронно через HTTP REST. Каждый сервис, требующий авторизации, отправляет Bearer-токен клиента на эндпоинт GET /internal/validate сервиса auth-service через библиотеку httpx (async HTTP client).

Таблица 4 — Типы взаимодействия

Тип	Технология	Где применяется
Синхронный REST	HTTP + httpx (async)	Проверка JWT, валидация vacancy/resume при создании отклика
Маршрутизация	FastAPI reverse проху	API Gateway перенаправляет запросы к нужному сервису по префиксу URL
Auth propagation	Bearer JWT в заголовке Authorization	Клиент передаёт токен, каждый сервис проверяет его у auth-service

9. План перехода от монолита к микросервисам

Для миграции предлагается паттерн Strangler Fig, позволяющий постепенно вытеснять монолит без остановки работы приложения.

Этап 1. API Gateway. Разворачивается FastAPI-прокси, который изначально направляет все запросы `/api/v1/*` на монолит. Клиенты продолжают работать без изменений.

Этап 2. Выделение auth-service. Из монолита выносятся таблица users и эндпоинты аутентификации. Gateway начинает маршрутизировать `/auth/*` и `/users/*` на новый сервис (порт 8001). Добавляется эндпоинт `/internal/validate`.

Этап 3. Выделение vacancy-service. Переносятся таблицы skills, companies, vacancies, vacancy_skills и соответствующие эндпоинты. Gateway обновляет маршрутизацию для `/skills/*`, `/companies/*`, `/vacancies/*`.

Этап 4. Выделение resume-service. Переносятся таблицы resumes, work_experience, education и эндпоинты `/resumes/*`. Добавляется внутренний эндпоинт `/resumes/internal/{id}`.

Этап 5. Выделение application-service. Последняя часть монолита (applications) выносятся в отдельный сервис (порт 8004). Монолит полностью выводится из эксплуатации.

На каждом этапе обратная совместимость клиентского API сохраняется за счёт маршрутизации через Gateway.

Вывод

В результате выполнения ДЗ4 спроектирована микросервисная архитектура для приложения «Платформа поиска работы». Выделены четыре микросервиса (auth, vacancy, resume, application) по доменному принципу с соблюдением database-per-service. Определены синхронные HTTP-эндпоинты для межсервисного взаимодействия, специфицированные в формате OpenAPI. Разработан пошаговый план миграции с монолита по

паттерну Strangler Fig. Документ содержит все требования и шаги для последующей реализации микросервисов в рамках ЛР2.